



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Ingeniería en Computación

Materia: Estructura de Datos y Algoritmos II

Proyecto: Visualizador Interactivo de Algoritmos

Documentación Técnica de Implementación de Algoritmos y Estructuras de Datos

Volumen I - Código Fuente y Referencia Técnica

Equipo de Desarrollo - Backend:

- Francisco José Coutiño Morales (*Lead Backend Developer*)
- Ernesto Flamenco Villaseñor (*Backend Developer*)
- Tlalli Ortega González (*Backend Developer*)
- Sergio Alonso Salcedo Sainz (*Backend Developer*)
- Orlando Ocampo Lagunas (*Backend Developer*)
- Aarón Salazar Castillo (*Backend Developer*)
- Erick Cruz Solís (*Backend Developer*)
- Augusto Gandarilla Ibarra (*Backend Developer*)
- Neri Israel Cruz Cárdenas (*Backend Developer*)

Periodo: 2026 – 2

Índice

1. Algoritmos de Ordenamiento	2
1.1. Bubble Sort	2
1.2. Heap Sort	3
1.3. Quick Sort	3
1.4. Counting Sort	4
1.5. Radix Sort	5
1.6. Merge Sort	6
2. Algoritmos de Búsqueda	7
2.1. Búsqueda Lineal	7
2.2. Búsqueda Binaria	7
2.3. Tabla Hash con Encadenamiento	8
3. Algoritmos de Grafos	8
3.1. Representación de Grafos	9
3.2. BFS — Búsqueda por Amplitud	9
3.3. DFS — Búsqueda en Profundidad	10
4. Árboles	10
4.1. Árbol Binario — Operaciones y Recorridos	10
4.2. Conversión de Notaciones Aritméticas	11
5. Algoritmos Paralelos	11
5.1. Reducción Paralela y Multiplicación de Matrices	11

1. Algoritmos de Ordenamiento

Los algoritmos de ordenamiento reorganizan una colección de elementos según un criterio de comparación. El módulo implementa seis algoritmos clásicos, cada uno con seguimiento de pasos intermedios para facilitar su visualización y análisis didáctico. Todos los endpoints se exponen bajo el prefijo `/api/sorting/`.

1.1. Bubble Sort

Algoritmo de ordenamiento *in-place* que compara pares adyacentes e intercambia sus posiciones cuando están fuera de orden. En cada pasada, el elemento máximo *burbujea* hacia su posición definitiva al final del arreglo.

Análisis de Complejidad:

Caso	Tiempo	Espacio
Mejor caso	$O(n)$	$O(1)$
Caso promedio	$O(n^2)$	$O(1)$
Peor caso	$O(n^2)$	$O(1)$

Archivo: `app/algoritmos/ordenamiento/sorting_algorithms.py`

```
1  def bubble_sort(arr: list) -> dict:
2      arr = arr.copy()
3      pasos = []
4      n = len(arr)
5      for i in range(n):
6          for j in range(0, n - i - 1):
7              if arr[j] > arr[j + 1]:
8                  arr[j], arr[j + 1] = arr[j + 1], arr[j]
9              pasos.append({
10                 "paso": len(pasos) + 1,
11                 "arreglo": arr.copy(),
12                 "comparando": [j, j + 1]
13             })
14      return {
15         "algoritmo": "Bubble Sort",
16         "resultado": arr,
17         "total_pasos": len(pasos),
18         "pasos": pasos
19     }
20
```

Listing 1: Implementacion de Bubble Sort

Endpoint Config:

POST `/api/sorting/bubble-sort`

Body: `{.arreglo": [int, ...]}`

Response: `{algoritmo, resultado, total_pasos, pasos}`

1.2. Heap Sort

Utiliza la estructura de datos *Max-Heap* para ordenar. Primero construye un heap máximo en $O(n)$ y luego extrae la raíz repetidamente, reconstruyendo el heap en $O(\log n)$ por extracción. Garantiza $O(n \log n)$ en todos los casos.

Análisis de Complejidad:

Caso	Tiempo	Espacio
Mejor caso	$O(n \log n)$	$O(1)$
Caso promedio	$O(n \log n)$	$O(1)$
Peor caso	$O(n \log n)$	$O(1)$

```
1  def heap_sort(arr: list) -> dict:
2      arr = arr.copy()
3      pasos = []
4      n = len(arr)
5      # Construccion del Max-Heap (Floyd)
6      for i in range(n // 2 - 1, -1, -1):
7          _heapify(arr, n, i, pasos)
8      # Extraccion elemento a elemento
9      for i in range(n - 1, 0, -1):
10         arr[0], arr[i] = arr[i], arr[0]
11         pasos.append({"paso": len(pasos)+1, "arreglo": arr.copy()})
12         _heapify(arr, i, 0, pasos)
13     return {"algoritmo": "Heap Sort", "resultado": arr,
14            "total_pasos": len(pasos), "pasos": pasos}
15
16     def _heapify(arr, n, i, pasos):
17         mayor = i
18         izq = 2 * i + 1
19         der = 2 * i + 2
20         if izq < n and arr[izq] > arr[mayor]: mayor = izq
21         if der < n and arr[der] > arr[mayor]: mayor = der
22         if mayor != i:
23             arr[i], arr[mayor] = arr[mayor], arr[i]
24             pasos.append({"intercambio": [i, mayor], "arreglo": arr.copy()})
25             _heapify(arr, n, mayor, pasos)
26
```

Listing 2: Implementacion de Heap Sort

Endpoint Config:

POST /api/sorting/heap-sort

Body: {".arreglo": [int, ...]}

1.3. Quick Sort

Algoritmo divide y vencerás que selecciona un *pivote*, particiona el arreglo y aplica el proceso recursivamente. Excelente rendimiento práctico gracias a la localidad de referencia y bajo overhead de memoria.

Análisis de Complejidad:

Caso	Tiempo	Espacio
Mejor caso	$O(n \log n)$	$O(\log n)$
Caso promedio	$O(n \log n)$	$O(\log n)$
Peor caso	$O(n^2)$	$O(n)$

```
1 def quick_sort(arr: list) -> dict:
2     arr = arr.copy()
3     pasos = []
4     _quick_sort_recursivo(arr, 0, len(arr) - 1, pasos)
5     return {"algoritmo": "Quick Sort", "resultado": arr,
6           "total_pasos": len(pasos), "pasos": pasos}
7
8 def _partition(arr, bajo, alto, pasos):
9     pivote = arr[alto]
10    i = bajo - 1
11    for j in range(bajo, alto):
12        if arr[j] <= pivote:
13            i += 1
14            arr[i], arr[j] = arr[j], arr[i]
15    pasos.append({"arreglo": arr.copy(), "pivote": alto,
16              "intercambio": [i, j]})
17    arr[i + 1], arr[alto] = arr[alto], arr[i + 1]
18    pasos.append({"arreglo": arr.copy(), "pivote_colocado": i + 1})
19    return i + 1
20
```

Listing 3: Implementacion de Quick Sort

Endpoint Config:

POST /api/sorting/quick-sort

Body: {".arreglo": [int, ...]}

1.4. Counting Sort

Algoritmo no comparativo. Cuenta ocurrencias, acumula conteos y construye el arreglo de salida. Rompe la cota matemática al explotar información adicional sobre el rango de valores.

Análisis de Complejidad:

Caso	Tiempo	Espacio
Todos los casos	$O(n + k)$	$O(k)$

```
1 def counting_sort(arr: list[int]) -> dict:
2     arr = arr.copy()
3     steps = [{"array": arr.copy(), "descripcion": "Array inicial"}]
```

```

4     maximo = max(arr)
5     minimo = min(arr)
6     rango = maximo - minimo + 1
7     conteo = [0] * rango
8     for num in arr:
9         conteo[num - minimo] += 1
10    steps.append({"conteo": conteo.copy(),
11                "descripcion": "Conteo de ocurrencias"})
12    for i in range(1, rango):
13        conteo[i] += conteo[i - 1]
14    salida = [0] * len(arr)
15    for num in reversed(arr):
16        salida[conteo[num - minimo] - 1] = num
17        conteo[num - minimo] -= 1
18    return {"sorted_array": salida, "steps": steps,
19           "complexity": {"best": "O(n+k)", "worst": "O(n+k)",
20                          "space": "O(k)"}
21

```

Listing 4: Implementacion de Counting Sort

Endpoint Config:

POST /api/sorting/counting-sort

Body: {".array": [int, ...]}

1.5. Radix Sort

Ordena dígito a dígito (LSD a MSD) usando Counting Sort estable como subrutina. Para números con d dígitos en base b : $O(d(n+b))$. Cuando d es constante y $b = O(n)$, logra $O(n)$.

```

1     def radix_sort(arr: list[int]) -> dict:
2         arr = arr.copy()
3         steps = [{"array": arr.copy(), "descripcion": "Array inicial"}]
4         maximo = max(arr)
5         exp = 1
6         while maximo // exp > 0:
7             arr = counting_sort_por_digito(arr, exp)
8             steps.append({"array": arr.copy(),
9                           "descripcion": f"Ordenado por exp={exp}"})
10            exp *= 10
11        return {"sorted_array": arr, "steps": steps,
12               "complexity": {"best": "O(nk)", "worst": "O(nk)",
13                              "space": "O(n+k)"}
14

```

Listing 5: Implementacion de Radix Sort

Endpoint Config:

POST /api/sorting/radix-sort

Body: {".array": [int, ...]}

1.6. Merge Sort

Divide el arreglo recursivamente en mitades y fusiona los resultados ordenados. Estable, garantiza $O(n \log n)$ en todos los casos. Estándar para ordenamiento externo (datos que no caben en RAM).

Análisis de Complejidad:

Caso	Tiempo	Espacio
Todos los casos	$O(n \log n)$	$O(n)$

```
1  def merge_sort_completo(arr: list[int]) -> dict:
2  arr = arr.copy()
3  steps = [{"array": arr.copy(), "descripcion": "Array inicial"}]
4  comparisons = [0]
5  sorted_arr = merge_sort(arr, steps, comparisons)
6  return {"sorted_array": sorted_arr, "steps": steps,
7         "comparisons": comparisons[0],
8         "complexity": {"best": "O(n log n)",
9                        "worst": "O(n log n)", "space": "O(n)"}]}
10
11 def merge_sort(arr, steps, comparisons):
12 if len(arr) <= 1:
13 return arr
14 mid = len(arr) // 2
15 L = merge_sort(arr[:mid], steps, comparisons)
16 R = merge_sort(arr[mid:], steps, comparisons)
17 return _merge(L, R, steps, comparisons)
18
19 def _merge(L, R, steps, comparisons):
20 resultado = []
21 i = j = 0
22 while i < len(L) and j < len(R):
23 comparisons[0] += 1
24 if L[i] <= R[j]:
25 resultado.append(L[i]); i += 1
26 else:
27 resultado.append(R[j]); j += 1
28 resultado.extend(L[i:])
29 resultado.extend(R[j:])
30 steps.append({"array": resultado.copy(), "L": L, "R": R})
31 return resultado
32
```

Listing 6: Implementacion de Merge Sort

Endpoint Config:

POST /api/sorting/merge-sort Body: {".array": [int, ...]}

GET /api/sorting/compare (Compara todos los algoritmos)

2. Algoritmos de Búsqueda

Los algoritmos de búsqueda localizan un elemento objetivo dentro de una estructura de datos. El módulo implementa búsqueda lineal, binaria y tabla hash, todos expuestos bajo el prefijo `/api/searching/`.

2.1. Búsqueda Lineal

Recorre la lista secuencialmente comparando cada elemento con el objetivo. No requiere que la lista esté ordenada. $O(1)$ mejor caso, $O(n)$ peor caso.

```
1 def busqueda_lineal(array, objetivo):
2     comparaciones = 0
3     for i, elemento in enumerate(array):
4         comparaciones += 1
5         if elemento == objetivo:
6             return {"found": True, "index": i,
7                     "comparisons": comparaciones,
8                     "complexity": "O(n)"}
9     return {"found": False, "index": -1,
10            "comparisons": comparaciones,
11            "complexity": "O(n)"}
12
```

Listing 7: Implementacion de Busqueda Lineal

Endpoint Config:

POST `/api/searching/linear`

Body: `{".array": [...], ".objetivo": valor}`

2.2. Búsqueda Binaria

Divide el espacio de búsqueda a la mitad en cada paso. Requiere arreglo ordenado. Complejidad $O(\log n)$: con 10^9 elementos, máximo 30 comparaciones.

```
1 def busqueda_binaria(array, objetivo):
2     izquierda = 0
3     derecha = len(array) - 1
4     comparaciones = 0
5     while izquierda <= derecha:
6         comparaciones += 1
7         medio = (izquierda + derecha) // 2
8         if array[medio] == objetivo:
9             return {"found": True, "index": medio,
10                    "comparisons": comparaciones,
11                    "complexity": "O(log n)"}
12         elif array[medio] < objetivo:
13             izquierda = medio + 1
14         else:
15             derecha = medio - 1
16     return {"found": False, "index": -1,
17            "comparisons": comparaciones, "complexity": "O(log n)"}
17
```


Listing 8: Implementacion de Busqueda Binaria

Endpoint Config:

POST /api/searching/binary

Body: {".array": [...], ".objetivo": valor}

2.3. Tabla Hash con Encadenamiento

Mapea claves a índices mediante función hash. Resuelve colisiones por encadenamiento. Factor de carga garantiza $O(1)$ promedio.

```

1      class TablaHash:
2      def __init__(self, tamaño=10):
3          self.tamaño = tamaño
4          self.tabla = [[] for _ in range(tamaño)]
5
6      def _hash(self, clave):
7          return hash(clave) % self.tamaño
8
9      def insertar(self, clave, valor):
10         idx = self._hash(clave)
11         for i, (k, v) in enumerate(self.tabla[idx]):
12             if k == clave:
13                 self.tabla[idx][i] = (clave, valor)
14                 return
15         self.tabla[idx].append((clave, valor))
16
17     def buscar(self, clave):
18         idx = self._hash(clave)
19         for k, v in self.tabla[idx]:
20             if k == clave:
21                 return {"found": True, "value": v, "index": idx}
22         return {"found": False, "value": None}
23

```

Listing 9: Tabla Hash con Encadenamiento

Endpoint Config:

POST /api/searching/hash

Body: {".clave": valor, ".tabla": {...}}

3. Algoritmos de Grafos

Un grafo consta de vértices V y aristas E . El módulo implementa dos representaciones y los dos recorridos fundamentales bajo /api/graphs/.

3.1. Representación de Grafos

```
1 def matriz_adyacencia(edges, n):
2     matriz = [[0] * n for _ in range(n)]
3     for u, v in edges:
4         matriz[u][v] = 1
5         matriz[v][u] = 1 # grafo no dirigido
6     return matriz
7
8 def lista_adyacencia(edges, n):
9     lista = {i: [] for i in range(n)}
10    for u, v in edges:
11        lista[u].append(v)
12        lista[v].append(u)
13    return lista
14
```

Listing 10: Matriz y Lista de Adyacencia

Endpoint Config:

POST /api/graphs/represent

Body: `{".edges": [[u,v],...], "n": int}`

3.2. BFS — Búsqueda por Amplitud

Explora nivel por nivel con cola FIFO. Garantiza camino más corto (en aristas) en grafos no ponderados. Complejidad $O(V + E)$.

```
1 from collections import deque
2
3 def bfs(grafo, inicio):
4     visitados = []
5     cola = deque([inicio])
6     vistos = set([inicio])
7     while cola:
8         nodo = cola.popleft()
9         visitados.append(nodo)
10        for vecino in grafo[nodo]:
11            if vecino not in vistos:
12                vistos.add(vecino)
13                cola.append(vecino)
14        return visitados
15
```

Listing 11: Implementacion BFS

Endpoint Config:

POST /api/graphs/bfs

Body: `{".edges": [...], "n": int, inicio": int}`

3.3. DFS — Búsqueda en Profundidad

Explora profundizando en cada rama antes de retroceder. Implementado recursivamente. Complejidad $O(V + E)$.

```
1 def dfs(grafo, inicio, visitados=None):
2     if visitados is None:
3         visitados = []
4         visitados.append(inicio)
5     for vecino in grafo[inicio]:
6         if vecino not in visitados:
7             dfs(grafo, vecino, visitados)
8     return visitados
9
```

Listing 12: Implementacion DFS

Endpoint Config:

POST /api/graphs/dfs

Body: `{".edges": [...], "n": int, "inicio": int}`

4. Árboles

4.1. Árbol Binario — Operaciones y Recorridos

```
1 class Nodo:
2     def __init__(self, valor):
3         self.valor = valor
4         self.izquierdo = None
5         self.derecho = None
6
7     class ArbolBinario:
8         def __init__(self):
9             self.raiz = None
10
11         def insertar(self, valor):
12             if not self.raiz:
13                 self.raiz = Nodo(valor)
14             else:
15                 self._insertar(self.raiz, valor)
16
17         def _insertar(self, nodo, valor):
18             if valor < nodo.valor:
19                 if not nodo.izquierdo:
20                     nodo.izquierdo = Nodo(valor)
21                 else:
22                     self._insertar(nodo.izquierdo, valor)
23             else:
24                 if not nodo.derecho:
25                     nodo.derecho = Nodo(valor)
26                 else:
27                     self._insertar(nodo.derecho, valor)
```

```

28
29     def inorden(self, nodo, res=None):
30     if res is None: res = []
31     if nodo:
32     self.inorden(nodo.izquierdo, res)
33     res.append(nodo.valor)
34     self.inorden(nodo.derecho, res)
35     return res
36

```

Listing 13: Arbol Binario de Busqueda

4.2. Conversión de Notaciones Aritméticas

```

1     def infija_a_sufija(expresion: str) -> str:
2     precedencia = {'+': 1, '-': 1, '*': 2, '/': 2, '^': 3}
3     salida = []
4     pila = []
5     for token in expresion.split():
6     if token.isdigit() or token.isalpha():
7     salida.append(token)
8     elif token == '(':
9     pila.append(token)
10    elif token == ')':
11    while pila and pila[-1] != '(':
12    salida.append(pila.pop())
13    pila.pop()
14    else:
15    while (pila and pila[-1] != '(' and
16    precedencia.get(pila[-1], 0) >=
17    precedencia.get(token, 0)):
18    salida.append(pila.pop())
19    pila.append(token)
20    while pila:
21    salida.append(pila.pop())
22    return ' '.join(salida)
23

```

Listing 14: Infija a Sufija - Algoritmo de Shunting Yard

5. Algoritmos Paralelos

5.1. Reducción Paralela y Multiplicación de Matrices

```

1     import concurrent.futures
2     import math
3
4     def suma_paralela(arr: list) -> dict:
5     """W = O(n), D = O(log n) - modelo PRAM CREW"""
6     n = len(arr)
7     if n == 0:

```

```

8      return {"resultado": 0, "trabajo": 0, "profundidad": 0}
9      niveles = math.ceil(math.log2(n)) if n > 1 else 0
10     resultado = sum(arr)
11     return {
12         "resultado": resultado,
13         "trabajo": n,
14         "profundidad": niveles,
15         "eficiencia": n / (niveles * n) if niveles > 0 else 1,
16     }
17
18     def multiplicacion_matrices_paralela(A, B):
19         """W = O(n^3), Span = O(n)"""
20         n = len(A)
21         C = [[0] * n for _ in range(n)]
22         def calcular_fila(i):
23             for j in range(n):
24                 C[i][j] = sum(A[i][k] * B[k][j] for k in range(n))
25         with concurrent.futures.ThreadPoolExecutor() as executor:
26             executor.map(calcular_fila, range(n))
27         return C
28

```

Listing 15: Suma paralela y Multiplicacion